Копиращ конструктор и оператор=

титуляр на курса: д-р Тодор Цонков (ttsonkov@gmail.com)

практически примери

теория

От какво ще се състои настоящата лекция?

Копиращ конструктор и оператор=. Rule of Three.
Разделна компилация. Препроцесор. Статични член данни. Примери - класът Student.

Какво е копиращия конструктор?

Използва се при:
подаване по стойност, връщане по стойност, инициализация

Два механизма:

1)копиращ конструктор
2)оператор за присвояване (=)

Правилен вариант

```
class Student {
public:
    Student(const Student& other);
};
```

Извиква се само при създаване на нов обект
const – задължително
приема референция, не стойност

Кога се извиква копиращия конструктор?

```
Student s1("Ivan");
Student s2 = s1;   // copy constructor


Student s3(s1);    // copy constructor


void print(Student s) {}
print(s1); // copy constructor
```

Return Value Optimization

```
Student create() {
    Student s("Maria");
    return s; // copy ctor (или move /
RVO)
}

// Очаква се да върне копие, но RVO ще
оптимизира
// Задължително от C++ 17!

Student arr[2] = { s1, s1 };

//отново при инициализация се създава!
```

Проблем с shallow copy

Какво е плитко копие?

Плитко копие (shallow copy) е копиране на обект, при което:
се копират стойностите на членовете,
но ако обектът съдържа указатели (raw pointers), се копират само адресите, а не самите данни, към които сочат.

Лоши примери

```
class Student {
   char* name;
public:
   Student(const char* n) {
      name = new char[strlen(n)+1];
      strcpy(name, n);
   }
Student(const Student& other) {
   name = other.name; // pointer
copy
}
};
```

Компилаторът генерира плитко копие

Два обекта → един и същ name

❌ double delete

Правилен пример

```
Student(const Student& other) {
   name = new
char[strlen(other.name)+1];
   strcpy(name, other.name);
}

//Всеки студент е със собствен name
```

Какво е assignment operator=?

Student& operator=(const Student& other);
Използва се при вече съществуващи обекти

Трябва да:

1) освободи старите ресурси

2) копира новите

3) върне *this

Пример за operator=

```
Student& operator=(const Student& other)
{   if (this == &other) return *this;

    delete[] name;

    name = new char
      [strlen(other.name)+1];

    strcpy(name, other.name);

    return *this;
}
```

Rule of Three

Ако класът има:
деструктор
копиращ конструктор
оператор =

-> трябва да има и трите задължително

В следващата лекция ще се разгледа Rule of Five

Допълнение

Rule of Three важи за класове, които притежават ресурс:
динамична памет (new/delete), файл, mutex, сокет

Този ресурс трябва да се:

1) освобождава

2) копира коректно

3) присвоява коректно

Трите операции покриват трите различни момента от живота на обекта.

Разделна компилация

Какво е разделна компилация?

.h → декларации
.cpp → имплементация

Предимства:
по-бърза компилация
по-чист дизайн
независими модули

Student.h

```
#pragma once
#include <string>

class Student {
    std::string name;
public:
    Student(const std::string& n);
    std::string getName() const;
};
```

//В C++ 20 се въвеждат модулите
// за да решат проблеми - като multiple includes in headers

Student.cpp

```
#include "Student.h"
Student::Student(const std::string& n) :
name(n) {}

std::string Student::getName() const {
    return name;
}
```

Грешки при използване на header-и

```
// Student.h
class Student {
public:
    void print() const;
};

// Student.cpp
   void Student::print() {   // ❌ липсва
const
}
```

Добри практики

✔ .h = интерфейс
✔ .cpp = имплементация
✔ Минимални include-и
✔ Forward declarations
✔ Никакви .cpp includes
✔ Никакъв using namespace в header

```
-> Circular dependencies -> forward declaration
// A.h
#include "B.h"
class B;
class A {
    B b;
};

// B.h
#include "A.h"

class B {
    A a;
};
```

Какво е препроцесор?

Работи преди компилация
Основни директиви:
#include
#define
#ifdef / #ifndef

Условна компилация:

```
#ifdef DEBUG
std::cout << "Debug mode\n";
#endif
// полезна за дебъгване
```

Добри практики

```
#ifndef STUDENT_H
#define STUDENT_H
// код
#endif
};
```

```
#define PI 3.14        //лоша практика
constexpr double PI = 3.14; // добра практика
```

Обяснение на разделната компилация

Една C++ програма може да бъде разделена на множество изходни файлове (.cpp), които се компилират независимо един от друг – това се нарича разделна компилация.

Преди същинската компилация всеки изходен файл се обработва от препроцесора, който изпълнява всички директиви, започващи със символа #.

Например, при всяка среща на директивата #include препроцесорът я заменя със съдържанието на съответния хедър файл, който обикновено съдържа декларации.

В резултат на компилацията на всеки .cpp файл се получава отделен обектен файл (с разширение .obj), съдържащ машинен код.

Изпълнимият файл на програмата (с разширение .exe) се създава на следващ етап – свързване (linking) – при което линкерът обединява всички обектни файлове.

Линкерът свързва всички референции към имена (променливи, функции, класове и др.) от даден обектен файл със съответните им дефиниции, които могат да се намират в други обектни файлове.

Възможно е дадена дефиниция да липсва във всички обектни файлове. В такъв случай линкерът я търси в стандартната C++ библиотека, стандартната C библиотека, както и във всички допълнително указани от програмиста библиотеки. Ако дефиницията не бъде открита никъде, линкерът генерира грешка.

Какво е CMake?

CMake е инструмент, който генерира build файлове
(Makefile, Visual Studio, Ninja и др.) от билд конфигурация.

❓ Защо не компилираме директно с g++?
1)проектът има много .cpp файлове

2)различни платформи (Linux / Windows / macOS)

3)различни компилатори

Основна идея

➡️ Описваме проекта веднъж (CMakeLists.txt)
➡️ CMake генерира подходящ build
➡️ компилаторът върши останалото

```
project/
  ├── CMakeLists.txt
  ├── main.cpp
  ├── Student.h
  └── Student.cpp
```

Какво са статични член данни?

Член-променлива, споделена между всички инстанции на класа.

Декларира се в класа; дефинира се (и, при нужда, инициализира) извън него — с изключение на inline static (C++17+).

Статичните член-данни имат живот, различен от обектите: инициализират се преди main() или при първия използван translation unit (implementation-defined ordering).

```cpp
class MyClass {
  public:
  static int s_count; // декларация
  MyClass();
};


// source MyClass.cpp
int MyClass::s_count = 0; // дефиниция + инициализация


struct Widget {
  inline static int s_count = 0;
  // C++17+: дефиниция в header
  static constexpr int MAX = 256;
  //константа известна на компилатора
};

//constexpr - смята се по време на компилация
```

Проблеми при статичните данни

Редът на инициализация на статични обекти в различни translation units е неопределен.

Това води до бъгове, когато един статичен обект използва друг статичен обект от друг .cpp файл.

```
extern int valueB;
int valueA = valueB + 1; // може да е неинициализирана

// B.cpp
int valueB = 42;
```

Решение

```
int& safeValue() {
    static int value = 42; // lazy + thread-safe
    return value;
}
//Scott Meyers - пример за Singleton
```

Практически съвети

Избягвайте глобални и статични обекти със сложна логика.
Предпочитайте constexpr, inline static или function-local static.
Ако редът има значение — контролирайте го чрез функции.

Пример.

```cpp
#ifndef STUDENT_H
#define STUDENT_H

#include <cstring>
#include <iostream>

class Student {
private:
    char* name;             // динамичен ресурс
    int facultyNumber;

    static int studentCount;  // статичен член-данна

public:
    // 1) Конструктор по параметри
    Student(const char* name = "Unknown", int fn = 0);

    // 2) Копиращ конструктор  (Rule of Three)
    Student(const Student& other);

    // 3) Оператор за присвояване =  (Rule of Three)
    Student& operator=(const Student& other);

    // 4) Деструктор  (Rule of Three)
    ~Student();

    // getter-и
    const char* getName() const;
    int getFacultyNumber() const;

    // статичен метод
    static int getStudentCount();

    // помощен метод
    void print() const;
};

#endif // STUDENT_H
```

Пример.

```
#include "Student.h"

// дефиниция на статичния член
int Student::studentCount = 0;

Student::Student(const char* n, int fn)
    : facultyNumber(fn)
{
    name = new char[strlen(n) + 1];
    std::strcpy(name, n);
    studentCount++;
}

Student::Student(const Student& other)
    : facultyNumber(other.facultyNumber)
{
    name = new char[strlen(other.name) + 1];
    std::strcpy(name, other.name);
    studentCount++;
}

Student& Student::operator=(const Student& other) {
    if (this != &other) { // защита от само-присвояване
        delete[] name;

        name = new char[strlen(other.name) + 1];
        std::strcpy(name, other.name);

        facultyNumber = other.facultyNumber;
    }
    return *this;
}

Student::~Student() {
    delete[] name;
    studentCount--;
}

const char* Student::getName() const {
    return name;
}

int Student::getFacultyNumber() const {
    return facultyNumber;
}

int Student::getStudentCount() {
    return studentCount;
}

void Student::print() const {
    std::cout << "Student: " << name
              << ", FN: " << facultyNumber << '\n';
}
```

Въпроси?

Благодаря за вниманието!

Допълнителни материали: learncpp.com - глава 14, глава 21